assignment4

May 29, 2026

1 Engineering Modelling 2 - Assignment 4

Yaman Ashqar - a1884774

1.0.1 Q1(a)

- `education_score`: **numerical** since it is a continuous score.
- `income`: **categorical** because it takes one of three discrete levels: "Higher deprivation towns", "Mid deprivation towns", or "Lower deprivation towns".
- `out_of_work`: **numerical** since it is a continuous percentage value.

[ ]:
```
import pandas as pd
import statsmodels.formula.api as smf

# Load data
education = pd.read_csv('education.csv')
education = education[['education_score', 'income', 'out_of_work']].dropna()

# Check income levels
print(education['income'].unique())
print(f"Shape after dropping NA: {education.shape}")
```

```
['Mid deprivation towns' 'Higher deprivation towns'
 'Lower deprivation towns' 'Cities']
Shape after dropping NA: (638, 3)
```

[3]:
```
# Parallel slopes regression (no interaction effects)
# Set Cities as reference level
education['income'] = pd.Categorical(education['income'],
                        categories=['Cities', 'Higher deprivation towns',
                                    'Mid deprivation towns', 'Lower deprivation␣
 ↪towns'])

model_parallel = smf.ols('education_score ~ income + out_of_work',
                         data=education).fit()
print(model_parallel.summary().tables[1])
```

```
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
----------------------
```

```
                                          coef    std err          t      P>|t|
[0.025      0.975]
------------------------------------------------------------------------------
----------------------
Intercept                                0.1688      0.663      0.255      0.799
-1.133       1.471
income[T.Higher deprivation towns]       0.2270      0.576      0.394      0.694
-0.904       1.358
income[T.Mid deprivation towns]          1.3233      0.611      2.167      0.031
0.124       2.523
income[T.Lower deprivation towns]        3.2922      0.628      5.246      0.000
2.060       4.524
out_of_work                             -0.2358      0.031     -7.557      0.000
-0.297      -0.175
==============================================================================
======================
```

[4]:
```
# Separate regression (with interaction effects)
model_separate = smf.ols('education_score ~ income * out_of_work',
                         data=education).fit()
print(model_separate.summary().tables[1])
```

```
==================================================================================
==================================
                                                    coef    std err          t
P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
----------------------------------
Intercept                                        -0.8520      2.750     -0.310
0.757      -6.252       4.548
income[T.Higher deprivation towns]                1.3497      2.783      0.485
0.628      -4.114       6.814
income[T.Mid deprivation towns]                   1.2835      2.908      0.441
0.659      -4.427       6.994
income[T.Lower deprivation towns]                 4.6199      2.833      1.631
0.103      -0.943      10.183
out_of_work                                      -0.1451      0.239     -0.606
0.544      -0.615       0.325
income[T.Higher deprivation towns]:out_of_work   -0.0993      0.242     -0.411
0.681      -0.574       0.375
income[T.Mid deprivation towns]:out_of_work       0.0437      0.266      0.164
0.870      -0.479       0.566
income[T.Lower deprivation towns]:out_of_work    -0.1458      0.266     -0.549
0.583      -0.668       0.376
==================================================================================
==================================
```

1.0.2 Q1(b)

```
education['income'] = pd.Categorical(education['income'],
                        categories=['Cities', 'Higher deprivation towns',
                                    'Mid deprivation towns', 'Lower deprivation towns'])

model_parallel = smf.ols('education_score ~ income + out_of_work',
                         data=education).fit()
print(model_parallel.summary().tables[1])
```

Variable	coef	std err	t	P>|t|
Intercept	0.1688	0.663	0.255	0.799
income[T.Higher deprivation towns]	0.2270	0.576	0.394	0.694
income[T.Mid deprivation towns]	1.3233	0.611	2.167	0.031
income[T.Lower deprivation towns]	3.2922	0.628	5.246	0.000
out_of_work	-0.2358	0.031	-7.557	0.000

1.0.3 Q1(c)

```
model_separate = smf.ols('education_score ~ income * out_of_work',
                         data=education).fit()
print(model_separate.summary().tables[1])
```

Variable	coef	std err	t	P>|t|
Intercept	-0.8520	2.750	-0.310	0.757
income[T.Higher deprivation towns]	1.3497	2.783	0.485	0.628
income[T.Mid deprivation towns]	1.2835	2.908	0.441	0.659
income[T.Lower deprivation towns]	4.6199	2.833	1.631	0.103
out_of_work	-0.1451	0.239	-0.606	0.544
income[T.Higher deprivation towns]:out_of_work	-0.0993	0.242	-0.411	0.681
income[T.Mid deprivation towns]:out_of_work	0.0437	0.266	0.164	0.870
income[T.Lower deprivation towns]:out_of_work	-0.1458	0.266	-0.549	0.583

1.0.4 Q1(d)

For each level of `income`, the best-fit line is obtained by combining the intercept and slope terms from the interaction model.

Cities (reference level):

$$\widehat{\text{education\_score}} = -0.8520 + (-0.1451) \times \text{out\_of\_work}$$

$$= -0.8520 - 0.1451 \times \text{out\_of\_work}$$

Higher deprivation towns:

$$\widehat{\text{education\_score}} = (-0.8520 + 1.3497) + (-0.1451 - 0.0993) \times \text{out\_of\_work}$$

$$= 0.4977 - 0.2444 \times \text{out\_of\_work}$$

Mid deprivation towns:

$$\widehat{\text{education\_score}} = (-0.8520 + 1.2835) + (-0.1451 + 0.0437) \times \text{out\_of\_work}$$

$$= 0.4315 - 0.1014 \times \text{out\_of\_work}$$

Lower deprivation towns:

$$\widehat{\text{education\_score}} = (-0.8520 + 4.6199) + (-0.1451 - 0.1458) \times \text{out\_of\_work}$$

$$= 3.7679 - 0.2909 \times \text{out\_of\_work}$$

1.0.5 Q1(e)

The "extra term" in the separate regression model (compared to the parallel slopes model) is the interaction between `income` and `out_of_work`. i.e. the terms `income:out_of_work`. These allow the slope of `out_of_work` to differ across each level of `income`, rather than being constrained to be the same (parallel) across all levels.

1.0.6 Q1(f)

[5]:
```
# ANOVA comparison between parallel slopes and separate regression
from statsmodels.stats.anova import anova_lm

anova_results = anova_lm(model_parallel, model_separate)
print(anova_results)
```

```
   df_resid          ssr  df_diff    ss_diff         F    Pr(>F)
0     633.0  3602.164848      0.0        NaN       NaN       NaN
1     630.0  3592.123601      3.0  10.041248  0.587024  0.623676
```

```
from statsmodels.stats.anova import anova_lm

anova_results = anova_lm(model_parallel, model_separate)
print(anova_results)
```

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
Parallel slopes	633.0	3602.1648	—	—	—	—
Separate regression	630.0	3592.1236	3.0	10.0412	0.5870	0.6237

The ANOVA p-value is 0.6237, which is far above any conventional significance level (e.g. $\alpha = 0.05$). This means the interaction terms (the extra terms in the separate regression model) do not provide a statistically significant improvement in fit over the parallel slopes model. We therefore choose the parallel slopes regression model as our final model. It is simpler and the additional complexity of allowing separate slopes is not justified by the data.

1.0.7 Q2(a)

The problem is that `Pclass` contains the integer values 1, 2, and 3, so `statsmodels` treats it as a numerical variable and fits a single continuous slope across the three classes. However, `Pclass` is actually a categorical variable. The numbers 1, 2, and 3 are just labels for first, second, and third class, with no meaningful numeric spacing between them. Treating it as numerical incorrectly assumes that the difference in survival probability between first and second class is the same as between second and third class, which may not be true.

1.0.8 Q2(b)

The fix is to wrap `Pclass` in `C()` inside the formula string, which tells `statsmodels` to treat it as a categorical variable:

```
titanic_pclass_lr = glm('Survived ~ C(Pclass)', data=titanic,
                        family=Binomial()).fit()
```

The `C()` operator forces `statsmodels` to treat `Pclass` as a categorical variable without needing to modify the dataframe itself.

[6]:
```
import pandas as pd
from statsmodels.formula.api import glm
from statsmodels.genmod.families import Binomial

# Load Titanic data
titanic = pd.read_csv('titanic.csv')

# Q2(c): Logistic regression: Survived ~ Sex
model_sex = glm('Survived ~ Sex', data=titanic, family=Binomial()).fit()
print(model_sex.summary().tables[1])

# Q2(d): Odds ratio for female vs male
import numpy as np
coef_female = model_sex.params['Sex[T.male]']
odds_ratio = np.exp(coef_female)
print(f"\nOdds ratio (male vs female): {odds_ratio:.4f}")
print(f"Odds ratio (female vs male): {1/odds_ratio:.2f}")

# Q2(e): Probability of survival for males
intercept = model_sex.params['Intercept']
coef_male = model_sex.params['Sex[T.male]']
log_odds_male = intercept + coef_male
prob_male = 1 / (1 + np.exp(-log_odds_male))
print(f"\nLog-odds for male: {intercept:.4f} + {coef_male:.4f} = {log_odds_male:
  ↪.4f}")
print(f"Probability of survival for males: {prob_male:.2f}")
```

```
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
```

```
Intercept       1.0566      0.129      8.191      0.000       0.804       1.309
Sex[T.male]    -2.5137      0.167    -15.036      0.000      -2.841      -2.186
==============================================================================

Odds ratio (male vs female): 0.0810
Odds ratio (female vs male): 12.35

Log-odds for male: 1.0566 + -2.5137 = -1.4571
Probability of survival for males: 0.19
```

1.0.9 Q2(c)

```
titanic = pd.read_csv('titanic.csv')
model_sex = glm('Survived ~ Sex', data=titanic, family=Binomial()).fit()
print(model_sex.summary().tables[1])
```

Variable	coef	std err	z	P>|z|
Intercept	1.0566	0.129	8.191	0.000
Sex[T.male]	-2.5137	0.167	-15.036	0.000

The reference level is female. The negative coefficient for `Sex[T.male]` indicates that being male is associated with a significantly lower probability of survival.

1.0.10 Q2(d)

The odds ratio for female vs. male is obtained by exponentiating the negative of the `Sex[T.male]` coefficient:

$$\text{Odds ratio} = e^{-(-2.5137)} = e^{2.5137} = 12.35$$

The odds of surviving were 12.35 times higher for females than for males.

1.0.11 Q2(e)

The reference level is female, so for males we add both the intercept and the `Sex[T.male]` coefficient to get the log-odds, then apply the logistic function:

$$P(\text{survival} \mid \text{male}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1)}} = \frac{1}{1 + e^{-(1.0566 - 2.5137)}} = \frac{1}{1 + e^{-(-1.4571)}} = 0.19$$

The probability of survival for males was 0.19 (19%).

1.0.12 Q3

[7]:
```
import numpy as np

# Q3(a): Simulate m draws of 5 cards and count four-of-a-kinds
def simulate_four_of_a_kind(m, seed=42):
```

```
    # Build deck: 13 ranks x 4 suits = 52 cards
    # Represent each card by its rank (0-12), suit doesn't matter for four-of-a-kind
    deck = np.array([rank for rank in range(13) for suit in range(4)])

    rng = np.random.default_rng(seed)
    count = 0

    for _ in range(m):
        hand = rng.choice(deck, size=5, replace=False)
        # Check if any rank appears exactly 4 times
        _, freq = np.unique(hand, return_counts=True)
        if np.any(freq == 4):
            count += 1

    return count, count / m

# Q3(b): m = 10
count_10, prob_10 = simulate_four_of_a_kind(10)
print(f"m = 10:")
print(f"  Four-of-a-kinds: {count_10}")
print(f"  Simulated probability: {prob_10:.5f}")

# Q3(c): m = 100,000
count_100k, prob_100k = simulate_four_of_a_kind(100000)
print(f"\nm = 100,000:")
print(f"  Four-of-a-kinds: {count_100k}")
print(f"  Simulated probability: {prob_100k:.5f}")

# Q3(d): Compare with exact probability
exact = 624 / 2598960
print(f"\nExact probability: {exact:.5f}")
print(f"Error for m=10:     {abs(prob_10 - exact):.5f}")
print(f"Error for m=100000: {abs(prob_100k - exact):.5f}")
```

```
m = 10:
  Four-of-a-kinds: 0
  Simulated probability: 0.00000

m = 100,000:
  Four-of-a-kinds: 19
  Simulated probability: 0.00019

Exact probability: 0.00024
Error for m=10:     0.00024
Error for m=100000: 0.00005
```

1.0.13 Q3(a)

```
import numpy as np

def simulate_four_of_a_kind(m, seed=42):
    deck = np.array([rank for rank in range(13) for suit in range(4)])
    rng = np.random.default_rng(seed)
    count = 0
    for _ in range(m):
        hand = rng.choice(deck, size=5, replace=False)
        _, freq = np.unique(hand, return_counts=True)
        if np.any(freq == 4):
            count += 1
    return count, count / m
```

The deck is represented as 52 cards with ranks 0–12 (each repeated 4 times for the 4 suits). For each draw of 5 cards, we check whether any rank appears exactly 4 times using `np.unique`.

1.0.14 Q3(b)

```
count_10, prob_10 = simulate_four_of_a_kind(10)
print(f"Four-of-a-kinds: {count_10}")
print(f"Simulated probability: {prob_10:.5f}")
```

With $m = 10$: no four-of-a-kinds were observed in 10 draws, giving a simulated probability of **0.00000**. This is not a surprise since with such a small number of draws, it is very unlikely to observe a rare event with probability 0.024%.

1.0.15 Q3(c)

```
count_100k, prob_100k = simulate_four_of_a_kind(100000)
print(f"Four-of-a-kinds: {count_100k}")
print(f"Simulated probability: {prob_100k:.5f}")
```

With $m = 100{,}000$: 19 four-of-a-kinds were observed, giving a simulated probability of 0.00019.

1.0.16 Q3(d)

```
exact = 624 / 2598960
print(f"Exact probability: {exact:.5f}")
print(f"Error for m=10:     {abs(prob_10 - exact):.5f}")
print(f"Error for m=100000: {abs(prob_100k - exact):.5f}")
```

The exact probability from combinatorics is $624/2{,}598{,}960 = 0.00024$.

Simulation	Simulated probability	Error
$m = 10$	0.00000	0.00024
$m = 100{,}000$	0.00019	0.00005

As expected, $m = 100{,}000$ gives a simulated probability much closer to the true value. By the law

of large numbers, as the number of simulations increases, the simulated probability converges to the true probability. With only $m = 10$ draws, the estimate is highly unreliable, in this case it missed entirely, giving a probability of zero. With $m = 100{,}000$, the error is reduced by a factor of approximately 5, giving a much more accurate estimate of 0.00019 vs the true 0.00024.